

Methodology Memo — World Cup 2026 Match Forecasting

To: Project stakeholders (technical and non-technical) **From:** Modeling / engineering **Date:** 2026-06-13
Re: How the forecasting system produces, calibrates, benchmarks, and explains its pre-match probabilities

0. Executive summary

The system produces **pre-match probabilities** for every World Cup fixture: home win / draw / away win (1X2), each team's expected goals, the most likely scorelines, and derived markets (both-teams-to-score, over/under 2.5). It is built to be **well-calibrated and reproducible**, not to maximize headline "winners picked."

The pipeline is a four-stage assembly line:

1. **Rate every team's strength** from ~11,850 historical internationals (Elo).
2. **Convert strengths into a goal model** and expand it into a full scoreline grid (Dixon-Coles / Poisson).
3. **Calibrate** the raw probabilities so the stated confidence is honest.
4. **Grade** everything with proper scoring rules, and **benchmark** it against the betting market and an Elo-only baseline.

A fifth, read-only layer attaches a **deterministic plain-English explanation** to each forecast, derived entirely from the stored model quantities.

Every number shown anywhere is traceable to a stored database row or a saved model artifact. Nothing is computed at display time.

1. Data foundation

Plain English. The World Cup is only ~64 matches — far too few to judge how good each team is. So the *spine* of the system is the long history of international football; the tournament fixtures sit on top of that.

Specifics.

- **Historical internationals (~11,852 matches):** public, CC0-licensed dataset ([martj42/international_results](#)), loaded via [scripts/etl/pull_open_results.py](#). No API key required.
- **Tournament fixtures & results (68 World Cup 2026 fixtures):** football-data.org v4 API ([X-Auth-Token](#) header), via [scripts/etl/pull_fixtures.py](#). Idempotent upserts; undecided knockout slots ("Winner Group A") are skipped until teams are known.
- **Storage:** SQLite via SQLAlchemy 2.0 ([db/worldcup.sqlite](#)), schema portable to Postgres. Ten tables ([matches](#), [teams](#), [odds_snapshots](#), [predictions](#), [benchmark_predictions](#), [model_runs](#), [evaluation_metrics](#), ...). Raw payloads are immutable; odds are **append-only** (we never overwrite a historical price); model outputs are **versioned** by [model_run_id](#).

- **Provenance:** every ingested row carries `source`, `source_url`, `ingested_at`, `run_id`. Timestamps are UTC.
- **Anti-leakage:** features and training only ever use information available **before** kickoff (`src/features/build.py`).

2. Stage 1 — Team strength (Elo)

Plain English. Elo is the chess-style rating idea. Every team carries one number. When two teams play, the result that was *expected* (from the rating gap) is compared to what *actually happened*; the bigger the surprise, the bigger the rating move.

Specifics (`src/models/elo.py`).

- Update rule: $R' = R + K \cdot m \cdot (S - E)$, where $S \in \{1, 0.5, 0\}$ is the result, $E = 1 / (1 + 10^{-(\Delta R + H)/400})$ is the expected score, $K = 24$, base rating **1500**, and home bonus $H = 60$ Elo points — applied **only at non-neutral venues** (most World Cup games are neutral, so H is switched off).
- **Margin-of-victory damping:** $m = \ln(|\text{goal difference}| + 1)$. A 5–0 moves a rating more than a 1–0, but with diminishing returns, so freak blowouts don't distort ratings.
- **Elo → 1X2 benchmark:** rather than a bare win-expectancy, we fit a multinomial logistic regression of outcome on (`rating difference`, `home flag`), giving a calibrated three-way split. This is stored as the `elo_only` benchmark, **not** as the primary forecast.

3. Stage 2 — Goal model (Dixon-Coles) and the scoreline grid

This is the heart of the system, and it is what lets us output expected goals and scorelines rather than only win/draw/loss.

3a. The goal model

Plain English. Goals arrive somewhat randomly, like raindrops — you can't predict the exact count, but you can predict the *rate*. We give each team an **attack** dial (how many they tend to score) and a **defense** dial (how many they tend to concede), plus a shared baseline and a home bonus. A team's expected goals in a match = baseline + its attack – the opponent's defense (+ home bonus if not neutral).

Specifics (`src/models/dixon_coles.py`). The model is **log-linear**:

```
log(λ_home) = intercept + attack_home - defense_away + home_adv · (venue is not neutral)
log(λ_away) = intercept + attack_away - defense_home
```

- **Fitting:** weighted maximum likelihood, optimized with SciPy `L-BFGS-B` (`maxiter = 250`). Identifiability handled by centering attack/defense at zero each evaluation, plus a small **ridge penalty (1e-3)** that also stabilizes teams with few matches.
- **Time decay:** older matches count less, via exponential decay with a **half-life of 540 days** (a match that old counts half as much as a fresh one). This keeps the ratings current without throwing

away history.

- **Dixon-Coles low-score correction (ρ , bounded to ± 0.18):** plain Poisson slightly misprices the very common 0–0 / 1–0 / 0–1 / 1–1 results; ρ nudges exactly those four cells.
- **A note on the optimizer:** the numeric-gradient search reports `converged = False` because it hits a bounded iteration budget on the flat, regularized objective. This is benign — the expected-goals and probability outputs are stable well before the strict gradient tolerance would be met, which we verified by comparing against longer runs. The iteration count is logged for transparency.

3b. From rates to a full scoreline distribution

Plain English. "Expecting 1.8 goals" doesn't mean exactly 1.8 will be scored. We compute the probability of 0, 1, 2, ... goals for each side, combine them into a grid of every plausible scoreline, then read everything off that one grid.

Specifics (`src/models/scoreline.py`).

- Build the (11×11) matrix $P(\text{home} = x, \text{away} = y)$ (`max_goals = 10`) from two independent Poisson margins, apply the Dixon-Coles τ correction to the low-score block, and renormalize.
- Read off: **1X2** (sum the home-win / draw / away-win regions), **expected goals** (row/column means), **most likely scorelines** (top cells), **BTTS**, and **over/under 2.5**. All outputs come from one coherent object, so they are mutually consistent.

4. Stage 3 — Calibration (honest confidence)

Plain English. A model can pick winners and still lie about its confidence. Calibration asks: *when we say "70%," does it happen about 70% of the time?* If our 70%s only come true 60% of the time, we rein them in. This is the principle the whole project is built around.

Specifics (`src/models/calibration.py`). A multiclass calibrator (multinomial logistic regression on the log-probabilities) maps **raw** $\rightarrow$ **calibrated** 1X2 probabilities. We store **both** sets: `*_raw` (straight from the goal model) and `*_cal` (honesty-adjusted). The calibrated probabilities are what we display and trust. In scoring (Stage 4) we fit calibration **out-of-fold** so the evaluation is not flattered by training on its own test data.

5. Stage 4 — Benchmarking and evaluation

5a. Benchmarks

Plain English. We sanity-check ourselves against the betting market and a simple Elo-only baseline. The market is a *strong reference*, **not ground truth**.

Specifics (`src/evaluation/devig.py`). Bookmaker odds are padded so the three outcomes sum to more than 100% (the "overround"). We remove that padding two ways — **proportional** and **Shin** (which accounts for informed money) — to recover fair implied probabilities. The dashboard's **edge** column is simply *our probability minus the benchmark's*.

5b. Scoring rules

Plain English. We never grade ourselves on "how many winners we picked" — over 64 matches that's mostly luck. We use scores that reward *honest probabilities*.

Specifics ([src/evaluation/metrics.py](#)).

- **Log loss** — punishes confident wrong calls hardest (headline metric).
- **Brier score** — mean squared error between predicted probability and outcome.
- **Ranked Probability Score (RPS)** — football-aware: ordering matters, so predicting a win when it was a draw is penalized less than when it was a loss.
- **Reliability table** — the calibration check, plotted as a curve (dots on the diagonal = well calibrated).
- **Sharpness** — how decisive the forecasts are. The goal is to be as sharp as possible *while staying calibrated*.

5c. Backtest design

Plain English. We only let the model learn from matches that happened *before* the one it's predicting — never peeking at the future.

Specifics ([scripts/evaluation/backtest.py](#)). Rolling-origin (time-respecting) splits with out-of-fold calibration.

5d. Headline results (latest backtest)

Model	Log loss	Interpretation
Calibrated Dixon-Coles (dc_cal)	0.851	the production forecast
Elo-only benchmark	1.018	simple baseline
Uniform (1/3 each)	1.099	blind guessing

Lower is better. The calibrated goal model beats both baselines, and calibration both improves the scores and reduces overconfidence (sharpness). Sample real forecasts pass the smell test (e.g., Spain ~0.93 vs Cape Verde; Switzerland favored over Qatar).

6. Stage 5 — Qualitative reasoning (the "why")

Plain English. Each forecast comes with a short, plain-English explanation of *why* the model landed where it did — generated from the model's own numbers, not written by a language model. So the text is reproducible and every sentence traces to a quantity.

Specifics ([src/explain/reasons.py](#)). The key enabler is that Dixon-Coles is log-linear, so the home side's expected-goals edge decomposes **exactly**:

```
log( $\lambda_{\text{home}}$ ) - log( $\lambda_{\text{away}}$ )
= (attack_home - attack_away)      ← attacking edge
+ (defense_home - defense_away)    ← defensive edge
+ home_adv · (venue is not neutral) ← venue
```

Each forecast gets a headline plus up to six **ranked drivers**: **strength_gap** (Elo), **goals** (the exact split above + where each team's ratings sit in the field), **shape** (top scoreline, draw risk, tempo), **edge** (vs the Elo benchmark), **calibration** (raw → calibrated shift), and **uncertainty** (a thin-sample caveat when a team has few recent matches). The thresholds that turn numbers into words (e.g. what counts as a "clear favorite") are named constants in one file — they *are* the definitions. Stored in **predictions.reasoning_json**, versioned by **model_run_id**.

7. Reproducibility, testing, and provenance

- **One-command refresh** (**scripts/update.py**): pull → features → train → predict, ~45s, fully idempotent. **--skip-pull** and **--backtest** flags available.
 - **Read-only dashboard** (Streamlit, **app/dashboard/app.py**): renders only stored tables; never recomputes. Every panel is stamped with its **model_run_id** and timestamps.
 - **Tests**: 42 automated tests. Notably, the reasoning layer has a *rigor* test that asserts the goals decomposition equals the model's own **log($\lambda_{\text{home}}/\lambda_{\text{away}}$)** to machine precision, plus a reproducibility test (same prediction → byte-identical text).
 - **Versioning**: every model fit writes a **model_runs** row; predictions and benchmarks are versioned and never overwritten.
-

8. Assumptions, caveats, and what's next

Assumptions / caveats (stated plainly).

- Team strength is inferred from results only; **no lineup, injury, or xG inputs are currently in the model** — so the reasoning layer never cites them (that would be fiction). xG enrichment is scaffolded behind an adapter for later.
- Neutral-venue handling is applied throughout, but **knockout-specific dynamics** (extra time, penalties) are not yet modeled as a separate target.
- The market benchmark depends on odds being ingested; where odds are absent, only the Elo benchmark is shown.
- The optimizer's **converged = False** flag is benign (see §3a) but is called out for honesty.

Natural next steps.

- Add covariates (rest days, recent xG form) once xG ingestion is wired.
 - Model knockout progression (advance ≠ win-in-90) as a distinct output.
 - Accumulate live results through the tournament to refresh calibration and the backtest as real matches finish.
-

*All figures in this memo are reproducible from **db/worldcup.sqlite** (model run #5) via **python -m scripts.update**. File references point to the committed source.*